

Analyzing Selfish vs. Coordinated Routing in a Multi-Agent Congestion Game: An Implementation of Braess's Paradox

Malak Gehad
ID: 2305249

Ahd Osama
ID: 2305296

Omar Elsayed
ID: 2305057

Moataz Ahmed
ID: 2305223

Abstract—This paper presents a Python-based simulation of a multi-agent system competing over shared network resources, specifically modeling a Congestion Game. By implementing Braess's Paradox, we analyze the strategic behavior of agents navigating a network where edge latencies are load-dependent. We establish a simulation-based Nash Equilibrium via Best Response Dynamics and compare it against a manually optimized Social Optimum to quantify the Price of Anarchy. The findings demonstrate that purely selfish routing decisions lead to a sub-optimal global state, highlighting the inefficiencies inherent in uncoordinated multi-agent systems.

Index Terms—Game Theory, Multi-Agent Systems, Congestion Game, Braess's Paradox, Nash Equilibrium, Social Optimum

I. INTRODUCTION

In modern multi-agent systems, from traffic routing to cloud bandwidth allocation, independent entities frequently compete for shared resources. A fundamental challenge in these environments is that the cost of utilizing a resource increases with congestion.

This project explores this dynamic through the lens of a Congestion Game. We simulated up to 4000 autonomous agents navigating a directed graph from a source node to a destination node. The objective of this study is twofold: first, to computationally model how players iteratively update their strategies to minimize personal cost until a stable state (Nash Equilibrium) is reached; and second, to compare the efficiency of this selfish outcome against a centralized, coordinated outcome (Social Optimum). To clearly demonstrate the inefficiencies of selfish behavior, the network topology is designed around Braess's Paradox.

II. MATHEMATICAL MODEL

The congestion game is formalized as a tuple (N, E, Σ, C) , where:

- **Players (N):** A finite set of agents $i \in \{1, 2, \dots, n\}$, where n scales up to 4000 in our experiments.
- **Resources (E):** A set of edges (u, v) comprising a directed graph $G = (V, E)$.
- **Strategy Space (Σ):** The set of all simple paths P_i from the source node (S) to the destination node (D).

- **Cost Functions (C):** Each edge $e \in E$ has an associated latency function $c_e(x_e) = a \cdot x_e + b$, where x_e is the total number of agents traversing edge e , a is the congestion multiplier, and b is the base latency.

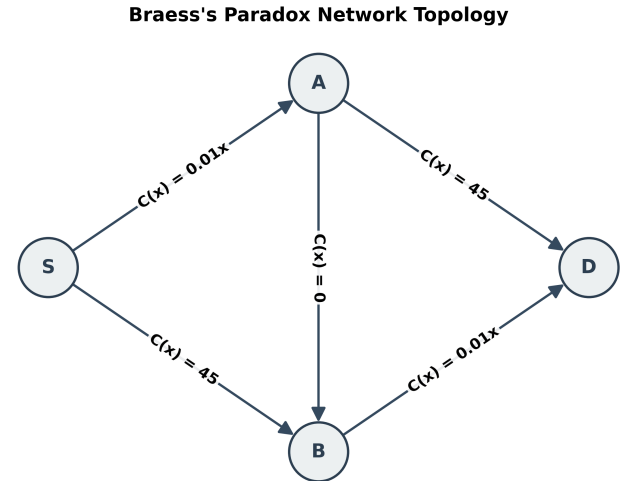


Fig. 1. The Braess's Paradox network topology utilized in the simulation. Edges denote flow direction, and functions represent congestion-dependent latency costs.

As illustrated in Fig. 1, the network features two highly congestion-sensitive routes ($S \rightarrow A$ and $B \rightarrow D$) and two static, high-capacity routes ($S \rightarrow B$ and $A \rightarrow D$). A zero-cost cross-link ($A \rightarrow B$) is introduced to simulate the paradox. The total cost incurred by agent i choosing path P is the sum of the edge costs: $C_i(P) = \sum_{e \in P} c_e(x_e)$.

III. CODE IMPLEMENTATION & REPRESENTATION

The simulation is implemented in Python using a modular, object-oriented architecture. The system logic is divided into distinct components: environment management, agent behavior, equilibrium simulation, and centralized optimization.

A. Agent and Environment Logic

The physical network is maintained by a `NetworkGraph` class, which utilizes a Depth-First Search (DFS) algorithm

with backtracking to dynamically discover all valid acyclic paths between the source and destination. Individual Agent objects possess memory of their current path and feature evaluation methods to calculate the hypothetical cost of switching paths given the current network load.

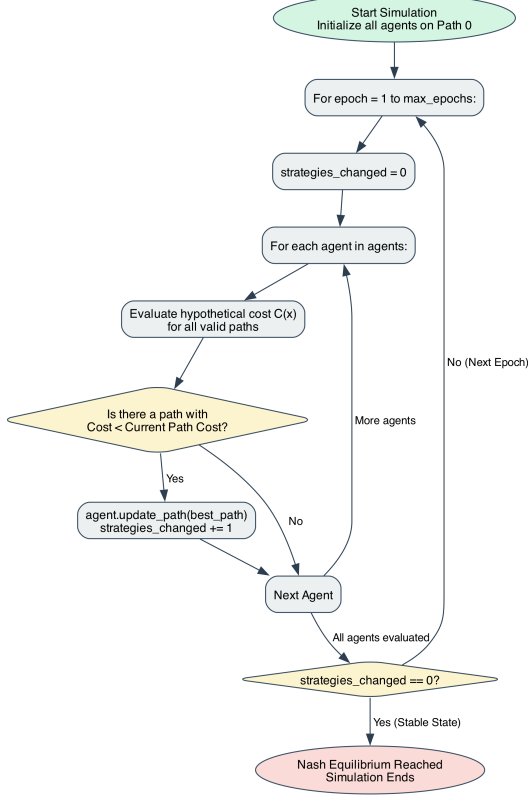


Fig. 2. Block diagram of the Best Response Dynamics loop implemented in the simulation.

B. Nash Equilibrium via Best Response Dynamics

To find the Nash Equilibrium computationally, we employed Best Response Dynamics, detailed in Fig. 2. In each epoch, agents evaluate the cost of all available paths. If an alternative path yields a strictly lower cost ($<$) than their current path, the agent switches. This iterative process continues until the system converges to a stable state where the number of strategy changes in an epoch equals zero.

C. Centralized Optimizer

For the mandatory efficiency comparison, a `SocialOptimumSolver` was implemented. Instead of minimizing individual cost, this algorithm calculates the marginal system cost of adding an agent to a specific path. Agents are assigned iteratively to the path that causes the least aggregate latency increase, yielding the coordinated optimum.

IV. RESULTS AND EVALUATION

Experiments were conducted to evaluate the impact of increasing the number of players and to quantify the systemic inefficiencies of selfish behavior.

A. Scalability and Convergence

The simulation successfully scaled to handle 4000 agents. Fig. 3 demonstrates the linear relationship between the total number of players and the average latency cost per agent.

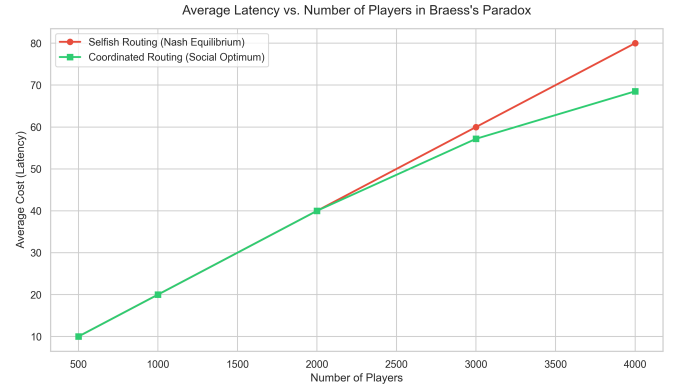


Fig. 3. Average latency cost compared between Selfish Routing and Coordinated Routing as the number of agents increases from 500 to 4000.

B. Efficiency Comparison

The results highlight a distinct divergence in system performance depending on the routing strategy. As depicted in Fig. 4, at maximum load (4000 players), the Selfish Routing algorithm reached a Nash Equilibrium where the average cost per player was 80.00. In contrast, the Coordinated Optimum achieved an average cost of 68.52.

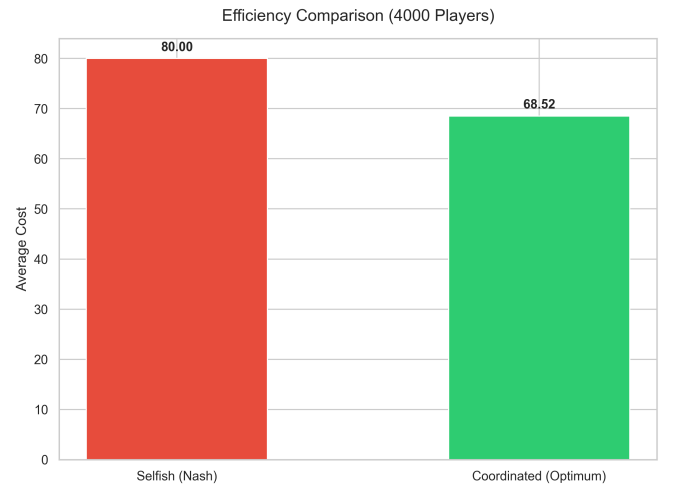


Fig. 4. Efficiency comparison showing the Price of Anarchy at maximum simulated load (4000 players).

V. INSIGHTS AND CONCLUSION

A. *Why Equilibrium Occurs*

The Nash Equilibrium occurs because the finite network space creates a localized minimum for individual cost. As agents flock to the zero-cost $A \rightarrow B$ cross-link, they inadvertently maximize the load on the $S \rightarrow A$ and $B \rightarrow D$ congestion-sensitive edges. Once all agents are suffering an equal, high latency, no single agent can unilaterally change their path without experiencing an equal or greater cost on the alternative routes.

B. *Efficiency of the System*

The outcome of the Nash Equilibrium is provably inefficient. The presence of the $A \rightarrow B$ shortcut acts as a "trap" for selfish agents. A purely self-interested player will always choose the path that appears immediately cheaper, failing to account for the negative externalities (congestion) they inflict on the rest of the network.

By contrast, the Social Optimum solver restricts access to the seemingly "cheap" cross-link, forcing traffic to balance evenly across the exterior paths. This coordination prevents the bottlenecks at nodes A and B, lowering the overall latency by approximately 14.3% for the entire system. Ultimately, this demonstrates that in shared-resource environments, uncoordinated selfish behavior degrades systemic utility.